Hands-on: Three Operator Cases

Run one notebook to understand three operator studies.

180 minutes / three short exercises / no coding experience required

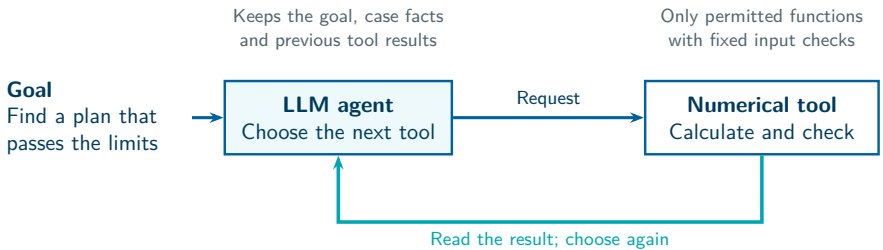
Alaa Selim
MIT/Dartmouth

Mohamed Chehada
UT Austin

Independent educational material / IEEE ComSoc theme adaptation

Source: Setup: COLAB_GUIDE.md; Classroom_Labs.ipynb.

What is agentic AI?



Grid teaching adaptation of the Stanford lecture, p.2. Model choices are recorded; calculations can run now.

- An LLM agent works toward a goal by choosing tools, reading their results and choosing what to do next.

Source: Monica Lam. CS224V Agentic AI, Lecture 1: Introduction. Stanford University, Fall 2026, p.2.

Stanford: tool calls become a workflow



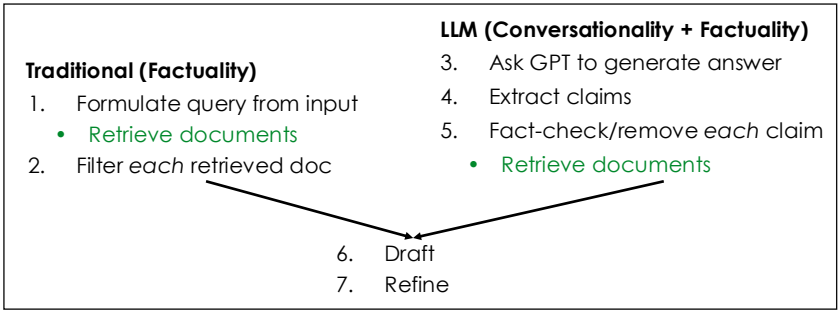
Original Stanford lecture excerpt, p.2. Next, connect tool use to evidence, task rules and decision checks.

Read the Stanford lecture, p.2

- A tool call runs one step. An agentic workflow links steps and uses their results to continue or stop.

Source: Monica Lam. CS224V Agentic AI, Lecture 1: Introduction. Stanford University, Fall 2026, p.2.

Stanford example: check before you write



Original Stanford lecture excerpt, p.19. WikiChat checks claims before drafting a reply.

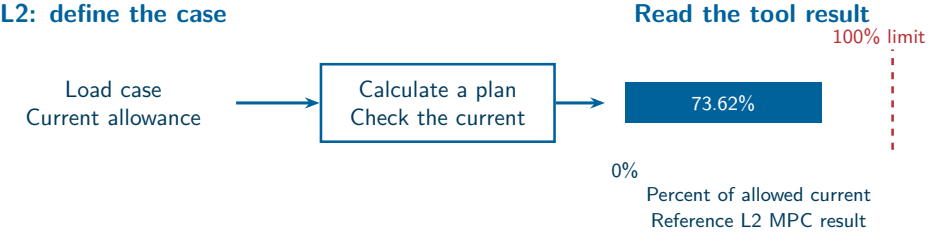
Read the Stanford lecture, p.19

- Grid lab takeaway: check the numerical result before writing the study summary.

Source: Monica Lam. CS224V Agentic AI, Lecture 1: Introduction. Stanford University, Fall 2026, p.19.

Voltage: a proposal needs a numerical check

Stanford idea / Colab preview: L2



This current check passes. Review voltage and model coverage too.

Our grid adaptation of WikiChat's checking habit. L2 calls Python directly; L10B replays saved Llama choices.

- L2 lets you test the numerical check. The saved L10B episode shows Llama choosing the next tool.

Source: Monica Lam. CS224V Agentic AI, Lecture 1: Introduction. Stanford University, Fall 2026, p.19; Classroom Labs L2.

Stanford example: tools need clear rules

- **Key Idea:** Full specifications are necessary
  - Not just functions to call!
- **Genie Worksheet: A Domain-Specific Language**
  - Specification of knowledge corpora + task workflow
    - Similar to workflow on webpages
  - Run-time to manage the long context

Original Stanford lecture excerpt, p.40. The lab adopts the idea of explicit rules in Python.

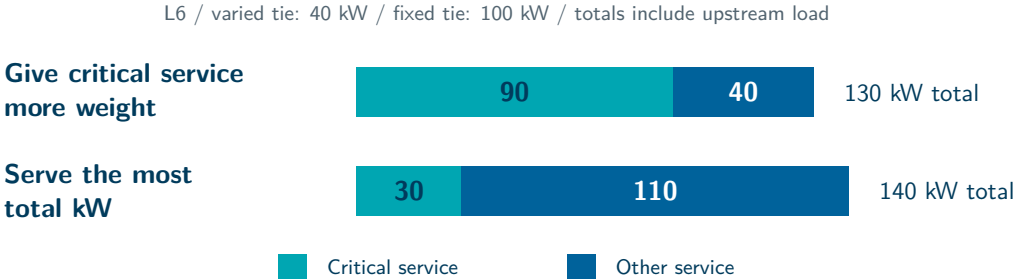
Read the Stanford lecture, p.40

- Write down the required inputs, permitted tools, limits and stop rules.
- Our lab keeps those rules in Python. A language reply cannot change the limits.

Source: Monica Lam. CS224V Agentic AI, Lecture 1: Introduction. Stanford University, Fall 2026, p.40.

Restoration: the service goal shapes the plan

Stanford idea / Colab preview: L6



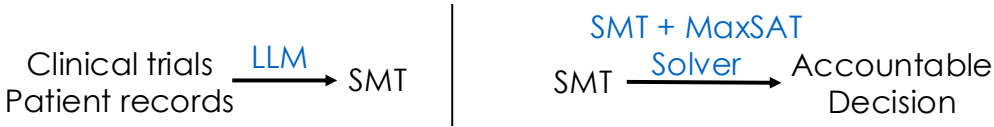
Our grid adaptation of explicit task rules. L6 uses the same 40 kW varied tie and 100 kW fixed tie for both goals.

- An agent must keep the stated service goal. L6 lets you see how that goal changes the plan.

Source: Monica Lam. CS224V Agentic AI, Lecture 1: Introduction. Stanford University, Fall 2026, p.40; Classroom Labs L6.

Stanford: language and decision checks

Stanford source example / grid teaching analogy



Clinical-trial research example. SMT represents logical constraints; MaxSAT helps find decision-changing conditions.

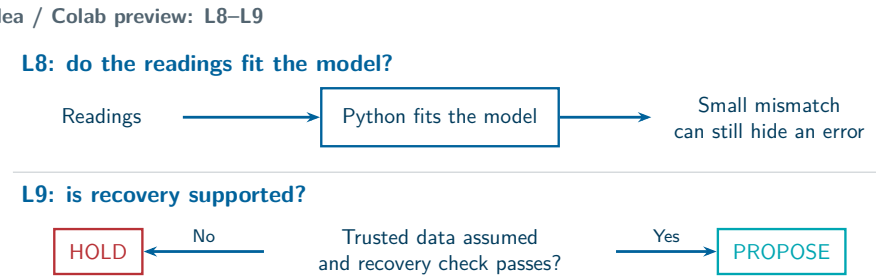
Read the Stanford lecture, p.53

- The lecture turns written facts into constraints, then uses a solver to check a decision.
- Our grid adaptation: Llama requests a study, Python checks its results, and the operator reviews the proposal.

Source: Monica Lam. CS224V Agentic AI, Lecture 1: Introduction. Stanford University, Fall 2026, p.53.

Cyber: a clear answer still needs evidence

Stanford idea / Colab preview: L8–L9



L9 assumes those extra readings are trusted. The numerical tool does not authenticate them.

Our grid adaptation of separating language from checks. The fixed L9 example returns HOLD or PROPOSE under stated assumptions.

- L8 tests whether readings can look consistent but be wrong. L9 adds a trust assumption and recovery checks.

Source: Monica Lam. CS224V Agentic AI, Lecture 1: Introduction. Stanford University, Fall 2026, p.53; Classroom Labs L8–L9.

Ask four questions about a study result

1. **Evidence**
   Which tool result supports the claim?
2. **Assumptions**
   Which inputs and model were used?
3. **Rules**
   Were the same limits and priorities kept?
4. **Next step**
   What does the operator still need to review?

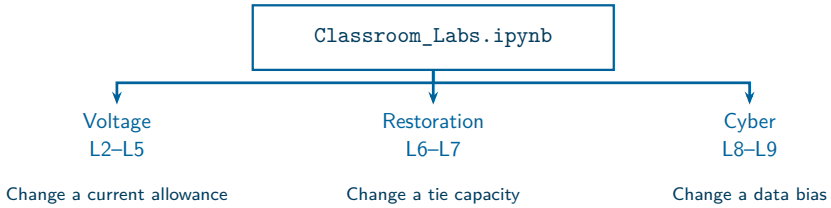
Grid teaching adaptation of Monica Lam's accountability concepts, Stanford CS224V introduction, pp.52–53.

Read the Stanford lecture, p.52

- Find the tool result, assumptions, rules and the operator's next step.
- If a claim has no supporting result, leave it unresolved and request the missing check.

Source: Monica Lam. CS224V Agentic AI, Lecture 1: Introduction. Stanford University, Fall 2026, pp.52–53.

One notebook covers all three cases



L10B replays Llama choices in the same file. Each case uses a separate teaching network.

Only Classroom Labs is required. The graph and trace notebooks are companion material.

Open Classroom Labs in Google Colab

- Save a copy in Drive. Keep this same notebook open for all three cases.

Source: Setup: COLAB_GUIDE.md; Classroom_Labs.ipynb.

Get ready in four small steps



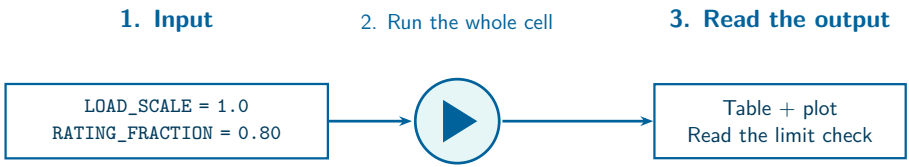
For the guided workshop, run each named lesson when prompted. Run all is a useful setup check.

Setup prepares the runtime. Then follow each named lesson cell with the slides.

- Open the notebook, save your copy, connect a CPU and run setup.
- Wait for Ready. Then run L0 and L1 before starting the three cases.

Source: Setup: COLAB_GUIDE.md; Classroom_Labs.ipynb.

Run a cell, then read what it returns



The play button executes Python. The saved text below a cell is its output.

Teaching illustration, not a Colab screenshot. Input lines match L2; run the full L2 cell.

- Find the L heading. Use the play button to run the whole matching code cell.
- Wait, then read the table or plot underneath it. Editing an input alone does not update output.

Source: Google Colab FAQ; Classroom Labs L2, cell classroom-007.

Run L0: choose the classroom route

Classroom Labs / L0 / classroom-002

Classroom_Labs.ipynb L0 (classroom-002), source lines 11–14.

```
RUN_OPENDSS = False       # True: fresh full-model audit +
   separate AC MPC solve in L5
RUN_LOCAL_LLAMA = False   # True: one optional local call in L11
ROOT = Path.cwd()
assert (ROOT / 'data/lindistflow3_network.json').exists(), 'Start
   Jupyter in the tutorial folder'
```

- Keep both switches False: new numerical checks, saved controller and archived model replies.
- If the data check fails, rerun setup before continuing.

Source: Classroom_Labs.ipynb, L0–L1; cells classroom-002 and classroom-004.

Run L1: does the evidence still match?

Classroom Labs / L1 / classroom-004

Executed notebook output

L1 · A tool contract is more than a prompt

classroom-004 · output 0 (zero-based) · recorded execution 3

	test	binding_pass
0	same request + same command	True
1	changed load; old evidence	False
2	changed command; old evidence	False

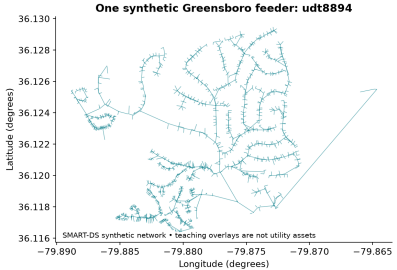
Reference L1 output: only the unchanged request and command match the evidence.

- Read binding_pass: does the record match the same inputs and command?
- Expect True, False, False. Changing the load or command breaks that match.

Source: Classroom_Labs.ipynb, L0–L1; cells classroom-002 and classroom-004.

Voltage: open L2

Classroom Labs / L2 / classroom-007

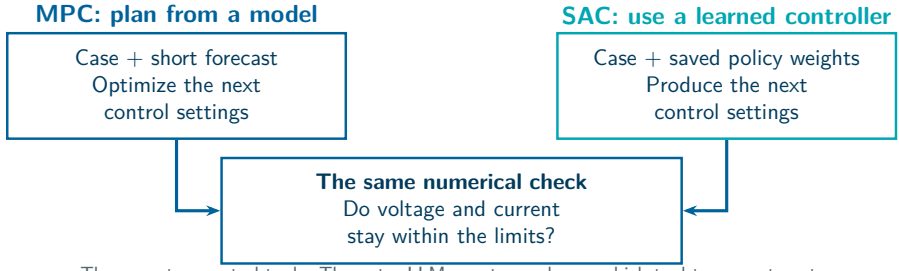


Synthetic SMART-DS teaching feeder; not an operating utility circuit.

- MPC makes a forecast-based plan. SAC is a saved learned controller.
- Check limits first. Then compare penalty score J; smaller is preferred after the checks pass.

Source: Classroom_Labs.ipynb, L2; cell classroom-007; src/continuous_control.py.

MPC plans; SAC proposes



These are two control tools. The outer LLM agent can choose which tool to request next.

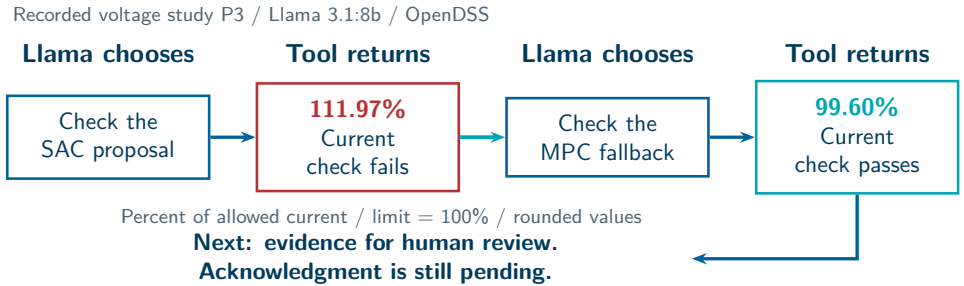
MPC: model predictive control. SAC: Soft Actor–Critic. These are control tools inside the larger workflow.

- Both tools propose control settings. The same numerical check tests their voltage and current limits.

Source: Classroom Labs notebook. Classroom_Labs.ipynb, L2; cell classroom-007; src/continuous_control.py. Public notebook and Python source. 24 October 2026; stable notebook cell IDs checked.

A real agent changes its next step

Recorded P3 example / separate from the L2 calculation



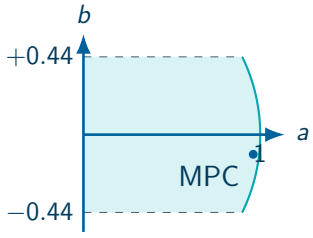
Recorded P3 example, separate from L2. 111.966607% and 99.604410% are rounded here.

- The failed tool result leads to a fallback check. Passing the checks still leaves human review.

Source: Workshop recorded model replies and numerical study records. Voltage P3_current_reserve: Llama 3.1:8b planner, recovery and review calls 1–3. Public teaching archive. 27 September 2026.

Control settings and limits

Classroom Labs / L2 / classroom-007



$$p_i = aS_i, \qquad q_i = bS_i$$

$$0 \le a \le 1, \quad |b| \le 0.44, \quad a^2 + b^2 \le 1$$

$b < 0$: reactive absorption

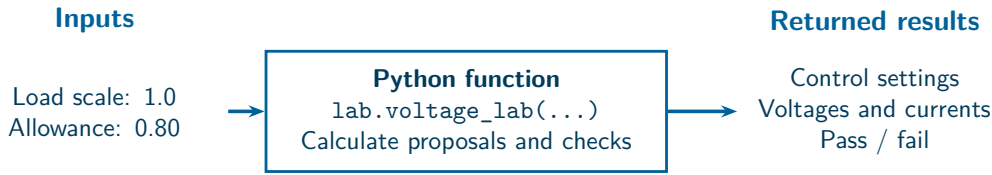
Inverter capability is one check.

Active and reactive settings share the inverter's power limit.

- a sets active-power participation; b sets reactive-power participation.
- Check the inverter limits, modeled voltages and allowed line current.

Source: Classroom_Labs.ipynb, L2; cell classroom-007; src/continuous_control.py.

A tool is a Python function in this lab



In L2, your cell calls this function directly.

In the recorded agent study, Python checks Llama's choice before running an allowed tool.

L2 calls lab.voltage_lab(LOAD_SCALE, RATING_FRACTION) directly. Recorded agent coordination is separate.

- A tool takes defined inputs and returns evidence. The notebook lets you see both.

Source: Classroom Labs notebook. Classroom_Labs.ipynb, L2; cell classroom-007; src/continuous_control.py. Public notebook and Python source. 24 October 2026; stable notebook cell IDs checked.

Run L2 with the default inputs

Classroom Labs / L2 / classroom-007

Classroom_Labs.ipynb L2 (classroom-007), source lines 1–4.

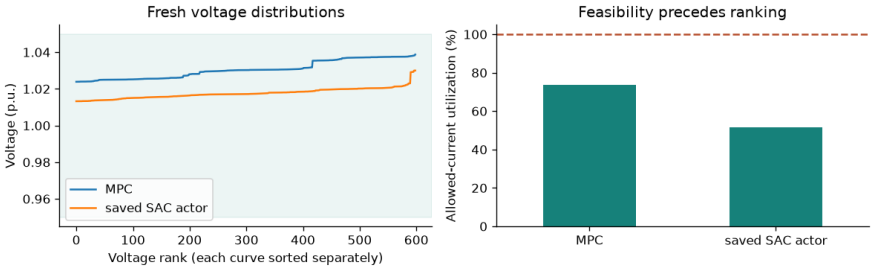
```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE, RATING_FRACTION)
```

- Run all of L2 to calculate the MPC plan and check the saved SAC controller's proposal.
- Read feasible, current_step_J and max_allowed_current_pct.

Source: Classroom_Labs.ipynb, L2; cell classroom-007; src/continuous_control.py.

Read the voltage and current plots

Classroom Labs / L2 / classroom-007



Reference L2 plots; rerun L2 to calculate plots for your inputs.

- Left: compare the sorted modeled voltages with the allowed band.
- Right: compare line current with the 100% allowed-current limit.

Source: Classroom_Labs.ipynb, L2; cell classroom-007; src/continuous_control.py.

Check limits before comparing scores

Classroom Labs / L2 / classroom-007

Executed notebook output

L2 · Model-based planning and model-free action proposals

classroom-007 · output 1 (zero-based) · recorded execution 4

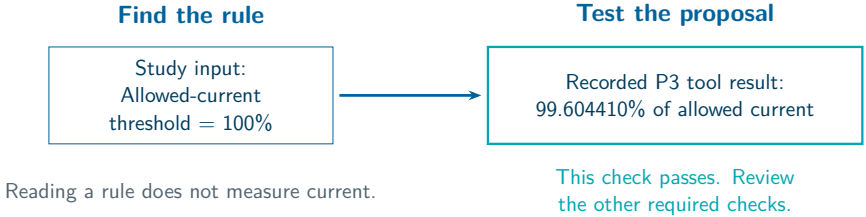
	method	feasible	current_step_J	a	b	vmin	vmax	max_allowed_current_pct	failures
0	MPC	True	0.031104	0.960307	-0.111114	1.023874	1.038709	73.619901	none
1	saved SAC actor	True	0.436428	0.340066	0.014024	1.013243	1.030000	51.679336	none

Reference L2 output with LOAD_SCALE=1.0 and RATING_FRACTION=0.80.

- Default MPC passes: current use 73.62%, penalty score J about 0.03110.
- Default saved SAC passes: current use 51.68%, J about 0.43643.

Source: Classroom_Labs.ipynb, L2; cell classroom-007; src/continuous_control.py.

A rule and a tool result answer different questions



Teaching analogy. The number shown comes from the recorded P3 replay, which is separate from L2.

- A document can tell us the limit. A numerical tool tells us how the proposal compares with it.
- Keep the source, the measured or calculated value, and the remaining checks together.

Source: Monica Lam. CS224V Agentic AI, Lecture 1: Introduction. Stanford University, Fall 2026, pp.17–25; recorded P3 current check.

Exercise 1: tighten the current allowance

Classroom Labs / L2 / classroom-007

```
RATING_FRACTION = 0.75
```

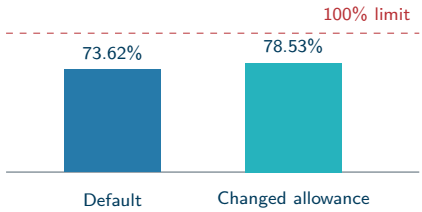
- In L2, change only RATING_FRACTION from 0.80 to 0.75. Predict, then rerun.
- For each method, report feasible, allowed-current use and current_step_J.
- Does a PASS cover the full medium- and low-voltage network? Explain.

Source: Classroom_Labs.ipynb, L2; cell classroom-007; src/continuous_control.py.

Change one number and explain the difference

L2: change one number

RATING_FRACTION
0.80 becomes 0.75
Predict, then rerun L2



Reference MPC results. Both checks still pass.

Reference L2 exercise: MPC uses 73.62% of the default allowance and 78.53% after the change.

- A tighter allowance makes the same current a larger percentage of that allowance.
- At 0.75, MPC uses 78.53% and SAC 54.85%. Both pass this particular test.

Source: Classroom Labs L2, cell classroom-007; verified default and exercise outputs.

Reset L2 before reading old model choices

Classroom Labs / L2 / classroom-007

Classroom_Labs.ipynb L2 (classroom-007), source lines 1–2.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
```

- Restore the default values and rerun L2.
- L3–L5 use the original test_01 case, separate from your changed L2 experiment.

Source: Classroom_Labs.ipynb, L2; cell classroom-007; src/continuous_control.py.

What runs now and what was recorded



Fresh calculation

Fresh numerical checks
Saved weights: inference



Actual recorded Llama

Old reply + original input
No new model call



Proposed control-room pilot

Advisory teaching design
Not a reported deployment

Three explicitly labeled teaching modes used throughout this workshop.

- The current cell calculates new results. The saved Llama reply keeps its original inputs.
- Changing your lab input does not rewrite the recorded agent study.

Source: Classroom Labs notebook. Setup: COLAB_GUIDE.md; Classroom_Labs.ipynb. Public notebook and Python source. 24 October 2026; stable notebook cell IDs checked.

L3 opens a separate recorded case

Classroom Labs / L3 / classroom-010

Classroom_Labs.ipynb L3 (classroom-010), source lines 1–2.

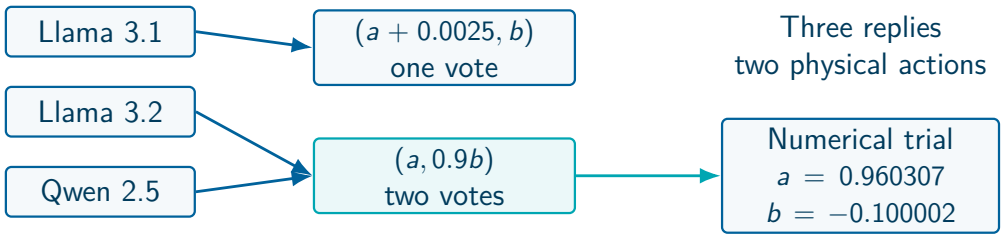
```
pool = lab.saved_pool('test_01')
display(pd.DataFrame(pool['responses'])[['model_id', 'operation_id
    ', 'reason', 'seconds', 'source']])
```

- Run L3. Read the original case ID, requested operations and recorded seconds.
- These reasons are old model replies. Displaying them makes no new model call.

Source: Classroom_Labs.ipynb, L3; cell classroom-010; archived test_01 context.

What did the models choose?

Classroom Labs / L3 / classroom-010



Three old replies request two distinct operations; two votes select reactive-power relaxation.

- Different wording can request the same tool and numbers.
- The notebook groups matching trials. An explanation alone does not show that limits pass.

Source: Classroom_Labs.ipynb, L3; cell classroom-010; archived test_01 context.

Run L4: check the old choice again

Classroom Labs / L4 / classroom-013

Classroom_Labs.ipynb L4 (classroom-013), source lines 5–9.

```
started = time.perf_counter()
selected_trials = lab.trial_saved_selection(pool)
timings['L4 new verification of archived choices'] = time.
   perf_counter() - started
display(selected_trials['table'])
assert pool['frozen'] and pool['training_updates'] == 24
```

- After L3, run L4. Python checks the selected proposal again for its original case.
- Compare majority vote with the saved LinUCB selector on that same case.

Source: Classroom_Labs.ipynb, L4; cell classroom-013; fresh checks of archived choices.

Separate the old choice from the new check

Classroom Labs / L4 / classroom-013

Executed notebook output

L4 · Learned response selection is a trial budget decision

classroom-013 · output 1 (zero-based) · recorded execution 6

	selector	operation	new_check_passes	current_step_J	verification_s	failures
0	majority vote	RELAX_Q_10_PERCENT	True	0.031082	0.091243	none
1	frozen LinUCB	RELAX_Q_10_PERCENT	True	0.031082	0.134089	none

Reference L4 output: old selector choices, new numerical checks of the original case.

- Both selectors choose the same reactive-power relaxation trial and pass the new MV check.
- Predicted reward is a learned score. Read new_check_passes for the engineering check.

Source: Classroom_Labs.ipynb, L4; cell classroom-013; fresh checks of archived choices.

Run L5: check what the model covers

Classroom Labs / L5 / classroom-016

Classroom_Labs.ipynb L5 (classroom-016), source lines 1–4.

```
started = time.perf_counter()
scope_audit = lab.backend_audit(pool, run_opendss=RUN_OPENDSS)
timings['L5 selected model scope checks'] = time.perf_counter() -
   started
display(scope_audit['table'])
```

- Keep RUN_OPENDSS=False. Run L5 and read the table and optional_mode message.
- L5 checks the original test_01 MV-MPC plan. OpenDSS is disabled.

Source: Classroom_Labs.ipynb, L5; cell classroom-016; MV-only default model.

A PASS covers only the model checked

Classroom Labs / L5 / classroom-016

Executed notebook output

L5 · Same physical action, different model coverage

classroom-016 · output 0 (zero-based) · recorded execution 7

	model	command	fresh	passes	vmin	vmax	current_pct	failures
0	lindistflow3	same archived MV-MPC first action	True	True	1.023874	1.038709	73.661582	none

Reference L5 output: a medium-voltage check with OpenDSS disabled.

- This check covers medium-voltage (MV) phases and currents, not all low-voltage (LV) equipment.
- For a full MV/LV request, get the wider check or HOLD.

Source: Classroom_Labs.ipynb, L5; cell classroom-016; MV-only default model.

Voltage: Llama asks to check the saved controller

Recorded agent example / separate P3 study

RECORDED REPLAY · llama3.1:8b · voltage · call 1

Input facts	Forecast load multipliers: [1.0, 0.94, 1.06]. Current allowance: 80% of nominal; voltage limits: [0.95, 1.05] p.u.
Llama choice	`VERIFY_SAC_FIRST`
Exact public reason	“A numerical SAC proposal is available; its fresh first-action check has not yet been requested.”
Recorded result	OpenDSS: 111.966607% of the allowed current; 2 current violations. Deterministic check: REJECT; failure = thermal.

Saved voltage example; separate from this case’s fresh calculation. Replay it in L10B later.

- Llama identifies the missing check. The current failure forces a new trial.

Source: Classroom Labs L10B: voltage replay, recorded call 1; full record in source guide.

Voltage: Llama chooses the existing fallback plan

Recorded agent example / separate P3 study

RECORDED REPLAY · llama3.1:8b · voltage · call 2	
Input facts	The supplied failed checks are: thermal. An MPC plan already exists; its recovery-eligibility failure list is empty.
Llama choice	`USE_MPC_PLAN`
Exact public reason	“The MPC plan has been generated and is eligible for a numerical check. This is the next allowed step in the recovery process.”
Recorded result	OpenDSS: 99.604410% of allowed current; no recorded violations. Proposed P = 2387.157 kW; Q = +325.738 kvar.

Saved voltage example; separate from this case’s fresh calculation. Replay it in L10B later.

- Llama chooses an existing MPC plan. Python checks whether the result passes.

Source: Classroom Labs L10B: voltage replay, recorded call 2; full record in source guide.

Voltage: the checks pass and human review remains

Recorded agent example / separate P3 study

RECORDED REPLAY · llama3.1:8b · voltage · call 3

Input facts	The full-feeder first-action check passed; voltage is 1.001783–1.035026 p.u. Protection, transients and implementation authority remain outside this model.
Llama choice	`REVIEW_EVIDENCE`
Exact public reason	“The model has converged and the numerical evidence indicates that the control actions are within the allowed limits. Reviewing the evidence will provide more insight into the model's performance and the selected control actions.”
Recorded result	Numerical status: MODEL_RECOMMEND; recorded human acknowledgment: PENDING. Acknowledged = False; dispatch authorized = False; equipment operated = False.

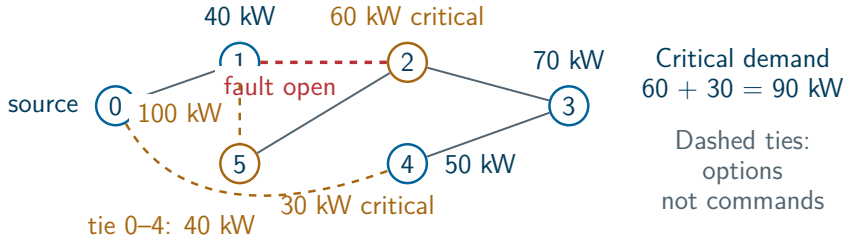
Saved voltage example; separate from this case’s fresh calculation. Replay it in L10B later.

- Human review remains pending. The fixed script returned the same numerical result.

Source: Classroom Labs L10B: voltage replay, recorded call 3; full record in source guide.

Restoration: open L6

Classroom Labs / L6 / classroom-018



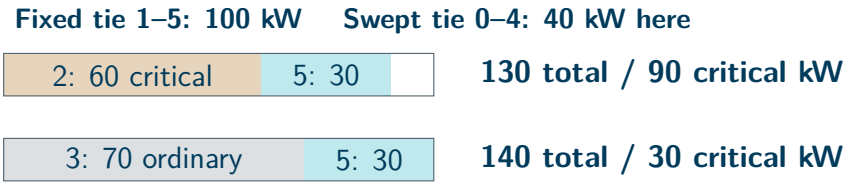
Six-node active-power teaching graph with whole-load pickup and one unavailable branch.

- Use the six-node teaching graph. Keep the faulted branch unavailable.
- Compare total restored load with restored critical load.

Source: Classroom_Labs.ipynb, L6; cell classroom-018; src/standard_formulations.py.

Choose the service goal before the plan

Classroom Labs / L6 / classroom-018



Different service goals can select different loads on the same network.

- The priority goal gives more weight to two critical loads.
- The comparison goal serves the most total kW, using the same network and pickup rules.

Source: Classroom_Labs.ipynb, L6; cell classroom-018; src/standard_formulations.py.

Run L6: change the tie capacity

Classroom Labs / L6 / classroom-018

Classroom_Labs.ipynb L6 (classroom-018), source lines 1–5.

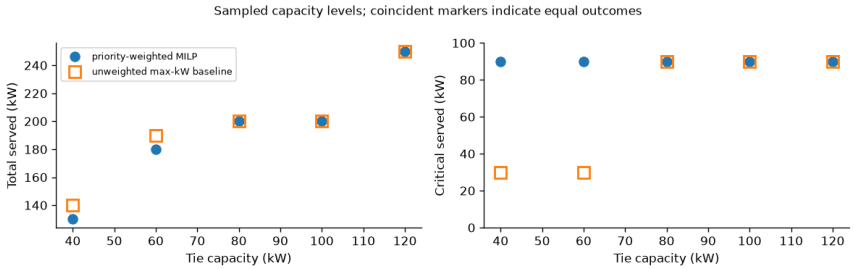
```
CAPACITIES_KW = (40., 60., 80., 100., 120.)
started = time.perf_counter()
restoration = lab.restoration_sweep(CAPACITIES_KW)
timings['L6 fresh MILP sweep + independent enumeration'] = time.
   perf_counter() - started
display(restoration['table'].drop(columns='pickup'))
```

- Run all of L6. MILP chooses whole loads; an exhaustive search checks its priority score.
- Read served_kw, critical_kw and MILP_matches_exhaustive_check.

Source: Classroom_Labs.ipynb, L6; cell classroom-018; src/standard_formulations.py.

Read both service plots

Classroom Labs / L6 / classroom-018



Reference L6 plots; rerun L6 to compare your chosen tie capacities.

- Left: total load served in kW. Right: critical load served in kW.
- Overlapping markers mean the two plans give the same result at that tested capacity.

Source: Classroom_Labs.ipynb, L6; cell classroom-018; src/standard_formulations.py.

The tradeoff at a 40 kW tie limit

Classroom Labs / L6 / classroom-018

Executed notebook output

L6 · Restoration: the objective expresses a service policy

classroom-018 · output 0 (zero-based) · recorded execution 8

Actual output header and rows 0–1 · tie_capacity_kw = 40.0

	tie_capacity_kw	method	served_kw	critical_kw	weighted_service	MILP_matches_exhaustive_check
0	40.0	priority-weighted MILP	130.0	90.0	940.0	True
1	40.0	unweighted max-kW baseline	140.0	30.0	410.0	True

Reference L6 results at a 40 kW tie limit: compare total and critical load served.

- Priority goal: 130 kW total, including 90 kW of critical load.
- Most-total-kW goal: 140 kW total, including 30 kW of critical load.

Source: Classroom_Labs.ipynb, L6; cell classroom-018; src/standard_formulations.py.

How Python checks the plan

Classroom Labs / L6 / classroom-018

MILP candidate
Loads + edges
priority score

→

Separate check
Fault open
source paths + tree
balance + capacities

→

Bind evidence
Request +
candidate
result + revision

Transport check only; AC operation and switching remain unverified.

Python checks the stated graph constraints, then compares the priority score independently.

- Check paths to a source, capacity limits, power balance and the faulted branch.
- Then compare the MILP service score with the independent exhaustive search.

Source: Classroom_Labs.ipynb, L6; cell classroom-018; src/standard_formulations.py.

Exercise 2: add more tie capacity

Classroom Labs / L6 / classroom-018

```
CAPACITIES_KW = (40., 60., 80., 100., 120., 180.)
```

- Add 180. to CAPACITIES_KW, keeping 40. first. Predict, then rerun L6.
- Compare total and critical load served at 40 and 180 kW.
- At which of these two capacities do the service goals give the same result?

Source: Classroom_Labs.ipynb, L6; cell classroom-018; src/standard_formulations.py.

Debrief: the tradeoff disappears here

Classroom Labs / L6 / classroom-018

250 kW total / 90 kW critical

- At 180 kW, both methods serve 250 kW total and 90 kW of critical load.
- Priority helps in the constrained case. It does not improve every case.

Source: Classroom_Labs.ipynb, L6; cell classroom-018; src/standard_formulations.py.

Reset the capacity list, then run L7

Classroom Labs / L7 / classroom-021

Classroom_Labs.ipynb L7 (classroom-021), source lines 1–3.

```
rest_record = restoration['records'][0]['weighted']
rest_request = {'task': 'restoration', 'capacity_kw':
   CAPACITIES_KW[0],
                'critical_nodes': [2, 5], 'model': '
   six_node_active_transport', 'revision': 1}
```

- Restore the original CAPACITIES_KW list and rerun L6.
- L7 checks the first 40 kW record, then changes the request to 20 kW.

Source: Classroom_Labs.ipynb, L7; cell classroom-021; request/evidence binding.

Read the evidence-match test

Classroom Labs / L7 / classroom-021

Executed notebook output

L7 · Restoration under a changed contract

classroom-021 · output 0 (zero-based) · recorded execution 9

```
Original contract: True
Changed rating with old evidence: False
Required response: HOLD old recommendation; request a fresh constrained solve.
```

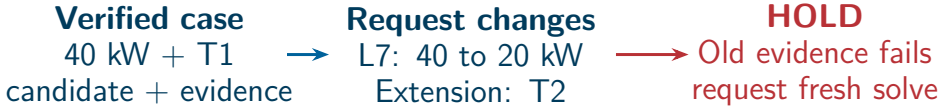
Reference L7 output: changed request, unchanged evidence, so a new check is needed.

- Original request: True. Changed rating with the old evidence: False.
- HOLD the old recommendation and request a new solve.

Source: Classroom_Labs.ipynb, L7; cell classroom-021; request/evidence binding.

Changed inputs need a new check

Classroom Labs / L7 / classroom-021



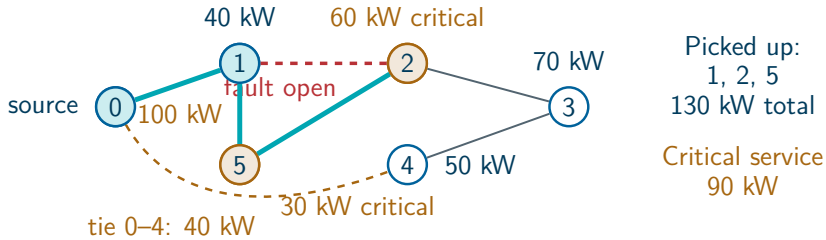
L7 changes capacity and revision; extra topology labels are teaching examples.

- "The evidence is old" does not mean "the changed plan fails its limits."
- Update the request, solve and check it, then match the new evidence to that request.

Source: Classroom_Labs.ipynb, L7; cell classroom-021; request/evidence binding.

Proposed agent role: keep the service goal

Classroom Labs / L6 / classroom-018



Proposed agent responsibilities; the current notebook cell runs Python.

- This cell runs Python: the operator sets the goal and limits; MILP chooses the whole loads.
- A proposed agent would request that solve and explain the returned tradeoff.

Source: Classroom_Labs.ipynb, L6; cell classroom-018; src/standard_formulations.py.

What this model does not check

Classroom Labs / L6 / classroom-018

- This active-power graph does not check AC voltage, reactive power or protection.
- The energized branches do not give the order of field switching steps.

Source: Classroom_Labs.ipynb, L6; cell classroom-018; src/standard_formulations.py.

Restoration: Llama chooses a useful new trial

Recorded agent example / separate feeder study

RECORDED REPLAY · llama3.1:8b · restoration · call 14	
Input facts	E01 failed its thermal check; E02 passed. E02 serves 1242.777 kW with 65.938897% tie loading.
Llama choice	`SHED_075`
Exact public reason	“The candidate E01 has a thermal failure, and the lowest cost passed candidate E02 has a cost of -5509.135240000002. Selecting SHED_075 will test 75% disconnection of the ordered noncritical load objects, which may improve the cost.”
Recorded result	SHED_075 produces E03: 1843.614 kW served and 474.040 critical kW. E03 passes at 91.867215% tie loading; cost = -6109.971300 (smaller is better).

Saved restoration example; separate from this case’s fresh calculation. Replay it in L10B later.

- The new trial serves more load, keeps critical service and passes the current check.

Source: Classroom Labs L10B: restoration replay, recorded call 14; full record in source guide.

Restoration: the next trial crosses the current limit

Recorded agent example / separate feeder study

RECORDED REPLAY · llama3.1:8b · restoration · call 15

Input facts	E03 is the lowest-cost passed candidate; one refinement evaluation remains. SHED_065 means disconnecting 65% of ordered noncritical load objects, not 65% of kW.
Llama choice	`SHED_065`
Exact public reason	“The candidate E01 has a thermal failure, and the lowest cost passed candidate E03 has a cost of -6109.9713, which is lower than the objective's target. Selecting SHED_065 will test 65% disconnection of the ordered noncritical load objects, which may improve the cost.”
Recorded result	E04 reaches 111.009413% tie loading: REJECT, thermal failure. The rejected trial has no eligible score; its larger served load cannot make it acceptable.

Saved restoration example; separate from this case’s fresh calculation. Replay it in L10B later.

- The next trial fails the current limit. Keep the previous passed candidate.

Source: Classroom Labs L10B: restoration replay, recorded call 15; full record in source guide.

Restoration: catch Llama's wrong ranking

Recorded agent example / separate feeder study

RECORDED REPLAY · llama3.1:8b · restoration · call 16	
Input facts	E02 cost = -5509.135240; E03 cost = -6109.971300; smaller is better. Only the passed candidates E03 and E02 are offered for ranking.
Llama choice	`E02, E03`
Exact public reason	"The candidates were ranked based on their scores, with the lowest score (best) first. E02 has a score of -5509.135240000002, and E03 has a score of -6109.971300000003."
Recorded result	Returned language order: E02 → E03. Correct numerical order: E03 → E02. The guard retains E03 and rechecks it as E05: 1843.614 kW served; no recorded failures.

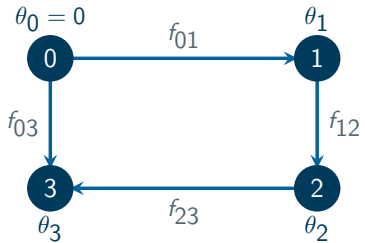
Saved restoration example; separate from this case's fresh calculation. Replay it in L10B later.

- The returned order is wrong. Python chooses the smaller cost and rechecks the plan.

Source: Classroom Labs L10B: restoration replay, recorded call 16; full record in source guide.

Cyber: open L8

Classroom Labs / L8 / classroom-023



$x = (\theta_1, \theta_2, \theta_3)^\mathsf{T}$

4 flows + 3 injections + 1 angle

8 readings; $\mathrm{rank}(H) = 3$

Residual degrees: $8 - 3 = 5$

Fixed linear DC model

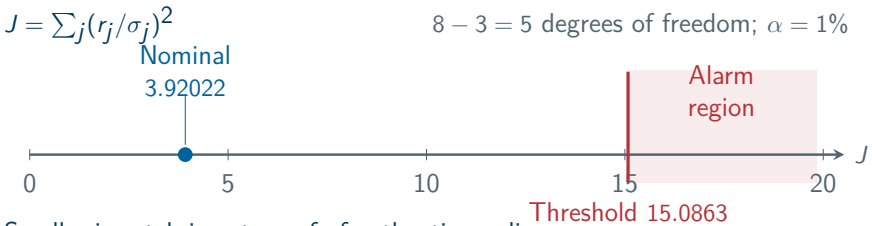
Four-bus linear DC teaching model; known synthetic truth is used only for offline comparison.

- Use the separate four-bus DC model and its supplied readings.
- Compare a residual alarm, which checks fit, with error against the known teaching truth.

Source: Classroom_Labs.ipynb, L8; cell classroom-023; four-bus DC fixture.

Best fit and true state are different

Classroom Labs / L8 / classroom-023



Small mismatch is not proof of authentic readings

A good fit to readings does not show that those readings are authentic.

- WLS finds the best fit to readings, giving more weight to lower stated uncertainty.
- Residual J measures fit mismatch. State error uses teaching truth unavailable to an operator.

Source: Classroom_Labs.ipynb, L8; cell classroom-023; four-bus DC fixture.

Run L8: compare two data attacks

Classroom Labs / L8 / classroom-023

Classroom_Labs.ipynb L8 (classroom-023), source lines 1–5.

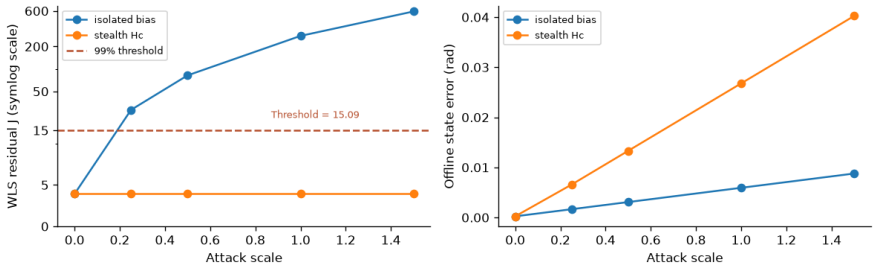
```
ATTACK_SCALES = (0., .25, .5, 1., 1.5)
started = time.perf_counter()
cyber = lab.cyber_sweep(ATTACK_SCALES)
timings['L8 fresh WLS attack sweep'] = time.perf_counter() -
   started
display(cyber['table'])
```

- Run all of L8 with its default scale list.
- Compare one biased reading with the coordinated Hc shift in the table and plots.

Source: Classroom_Labs.ipynb, L8; cell classroom-023; four-bus DC fixture.

Read both cyber plots

Classroom Labs / L8 / classroom-023



Reference L8 plots; rerun L8 to calculate results for your chosen attack scales.

- Left: residual J and its alarm threshold. Right: error against the known teaching truth.
- Follow the coordinated curve across both plots before calling a quiet alarm reassuring.

Source: Classroom_Labs.ipynb, L8; cell classroom-023; four-bus DC fixture.

At scale 1, the alarm misses a state error

Classroom Labs / L8 / classroom-023



Simple consistency illustration; the numerical values come from the eight-reading L8 model.

- One biased reading: J about 277.50, so the residual alarm is True.
- Coordinated Hc: J about 3.92, no alarm, but state error is about 0.02678 rad.

Source: Classroom_Labs.ipynb, L8; cell classroom-023; four-bus DC fixture.

Exercise 3: add a small isolated bias

Classroom Labs / L8 / classroom-023

```
ATTACK_SCALES = (0., .10, .25, .5, 1., 1.5)
```

- Add .10 to ATTACK_SCALES. Predict, then rerun L8.
- At scale 0.10, read J, the alarm flag and the error against known teaching truth.
- Does "no alarm" mean the reading is correct? Use both plots to explain.

Source: Classroom_Labs.ipynb, L8; cell classroom-023; four-bus DC fixture.

Debrief: no alarm does not prove truth

Classroom Labs / L8 / classroom-023

$10.14 < 15.09$ to no residual alarm

- At scale 0.10, the isolated-bias J is about 10.14, below the 15.09 threshold.
- A small bias can miss the threshold. A coordinated shift can preserve the fit.

Source: Classroom_Labs.ipynb, L8; cell classroom-023; four-bus DC fixture.

Reset L8 before the recovery example

Classroom Labs / L8 / classroom-023

Classroom_Labs.ipynb L8 (classroom-023), source lines 1–3.

```
ATTACK_SCALES = (0., .25, .5, 1., 1.5)
started = time.perf_counter()
cyber = lab.cyber_sweep(ATTACK_SCALES)
```

- Restore the original ATTACK_SCALES list and rerun L8.
- L9 uses a fixed recovery case, separate from the scale you changed.

Source: Classroom_Labs.ipynb, L8; cell classroom-023; four-bus DC fixture.

L9 uses a fixed recovery case

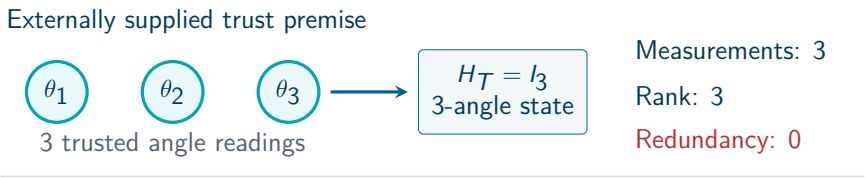
Classroom Labs / L9 / classroom-026

- Check the same case with and without replacement readings assumed to be trusted.
- Its control value does not follow your L8 scale change.

Source: Classroom_Labs.ipynb, L9; cell classroom-026; fixed trusted-recovery fixture.

Trusted readings are an assumption

Classroom Labs / L9 / classroom-026



Three trusted angle readings are assumed, not authenticated by this calculation.

- The example assumes three independent angle readings are trustworthy.
- The residual check and a model's explanation cannot authenticate those readings.

Source: Classroom_Labs.ipynb, L9; cell classroom-026; fixed trusted-recovery fixture.

Run L9: compare HOLD and PROPOSE

Classroom Labs / L9 / classroom-026

Classroom_Labs.ipynb L9 (classroom-026), source lines 1–2.

```
without_trust = lab.trusted_response(cyber, protected_available=
    False)
with_trust = lab.trusted_response(cyber, protected_available=True)
```

- Run the whole L9 cell, including its assertions.
- Read status first, then the proposed injection and the offline limit-violation count.

Source: Classroom_Labs.ipynb, L9; cell classroom-026; fixed trusted-recovery fixture.

Read the two decision states

Classroom Labs / L9 / classroom-026

Executed notebook output

L9 · Trusted state, constrained response, explicit HOLD

classroom-026 · output 0 (zero-based) · recorded execution 11

	condition	status	u_pu	offline_actual_violations
0	No trusted replacement channels	HOLD	NaN	NaN
1	Three trusted channels assumed	PROPOSE	0.121694	0.0

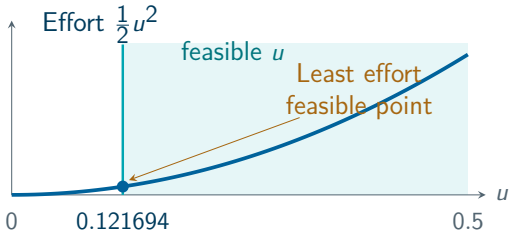
Reference L9 output for the fixed recovery case, not the edited L8 scale.

- No trusted replacement readings: HOLD, with no proposed injection.
- Trusted readings assumed: PROPOSE; u about 0.121694 p.u.; zero offline toy-model violations.

Source: Classroom_Labs.ipynb, L9; cell classroom-026; fixed trusted-recovery fixture.

Check the numerical correction

Classroom Labs / L9 / classroom-026



All four branches:

$|\hat{f}(u)| \leq \ell - 0.01$

$0 \leq u \leq 0.5$ p.u.

Fixed DC sensitivities

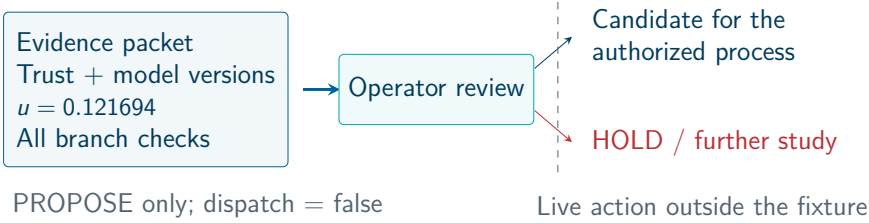
Fixed L9 correction: compare the predicted branch flow with the offline truth check.

- The QP solver minimizes effort, subject to branch limits and a 0.01 p.u. margin.
- Predicted flow12: 0.17000; offline actual: about 0.17046, below the 0.18000 rating.

Source: Classroom_Labs.ipynb, L9; cell classroom-026; fixed trusted-recovery fixture.

What makes the recovery possible?

Classroom Labs / L9 / classroom-026



Trusted information and numerical checks have separate roles.

- Assumed trusted readings support the estimate. The solver calculates a limited correction.
- The explanation must keep those conditions clear and leave the proposal for review.

Source: Classroom_Labs.ipynb, L9; cell classroom-026; fixed trusted-recovery fixture.

Cyber: Llama selects the tool already allowed

Recorded agent example / separate recovery study

RECORDED REPLAY · llama3.1:8b · cyber · call 7

Input facts	The structured task already authorizes WLS_TRUSTED_REDISPATCH. Trusted sensors are explicitly assumed; no live utility measurements are supplied.
Llama choice	`WLS_TRUSTED_REDISPATCH`
Exact public reason	“This tool is authorized for the given use case and scope, and it is designed for a specific DC redispatch example with trusted measurements.”
Recorded result	The numerical tool proposes u = 0.121693667 p.u.; predicted flow12 = 0.170000 p.u. Offline synthetic-truth flow12 = 0.170461 p.u.; rating = 0.180000 p.u.

Saved cyber example; separate from this case’s fresh calculation. Replay it in L10B later.

- The task supplies the tool and trust assumption. Python computes the correction.

Source: Classroom Labs L10B: cyber replay, recorded call 7; full record in source guide.

Cyber: a passing calculation still needs review

Recorded agent example / separate recovery study

RECORDED REPLAY · llama3.1:8b · cyber · call 8

Input facts	The model receives the actual numerical summary and passed_modeled_checks = True. No communication authentication, AC feeder or operating-permission check is included.
Llama choice	`REVIEW`
Exact public reason	“The provided JSON output indicates that the numerical workflow has completed successfully, and the results are within the expected scope. The 'passed_modeled_checks' field is set to True, indicating that the modeled checks have been passed. Therefore, the choice is to REVIEW the results to ensure they are accurate and within the expected range.”
Recorded result	Numerical verification: PASS. Language choice: REVIEW. Overall result: REVIEW. Human acknowledgment: PENDING; dispatch authorized = False; equipment operated = False.

Saved cyber example; separate from this case's fresh calculation. Replay it in L10B later.

- Keep REVIEW visible. The operator must resolve the trust assumption and missing checks.

Source: Classroom Labs L10B: cyber replay, recorded call 8; full record in source guide.

Run L10 before the Llama walkthrough

Classroom Labs / L10 / classroom-028

- Run L10 to inspect its separate archived five-stage study.
- Continue to L10B for the recorded Llama choices, then run L11 with its default False setting.

Source: Classroom_Labs.ipynb, L10–L11; cells classroom-028 and classroom-030.

Run L10B: watch Llama choose a tool

Classroom Labs / L10B / classroom-llama-walkthrough

Classroom_Labs.ipynb L10B (classroom-llama-walkthrough), source lines 1–5.

```
SCENARIO = "voltage"  # Change to "restoration" or "cyber", then
   rerun this cell.
from IPython.display import HTML, display
from src.llama_walkthrough import load_episodes,
   render_episode_html
replay_episodes = load_episodes(ROOT)
display(HTML(render_episode_html(replay_episodes[SCENARIO])))
```

- Now replay the three saved episodes: set SCENARIO to voltage, restoration and cyber, rerunning each time.

Source: Classroom_Labs.ipynb, L10B; cell classroom-llama-walkthrough; src/llama_walkthrough.py.

Finish L12: practice the readback

Classroom Labs / L12 / classroom-033

Classroom_Labs.ipynb L12 (classroom-033), source lines 9–10.

```
wrong_id = lab.acknowledge(mv_scope, 'wrong-id', 'Student operator
    ')
readback = lab.acknowledge(mv_scope, mv_scope['recommendation_id
    '], 'Student operator')
```

- Run all of L12 after completing L0–L11 in notebook order.
- Check wrong-ID rejection, simulated acknowledgment and equipment_dispatch=False.

Source: Classroom_Labs.ipynb, L12; cell classroom-033; simulated readback only.

Read the two readback outcomes

Classroom Labs / L12 / simulated readback

Executed notebook output

L12 · Operator handoff: acknowledge the recommendation you actually checked

classroom-033 · output 0 (zero-based) · recorded execution 14

Contiguous columns: state through equipment_dispatch; original row order and row index.

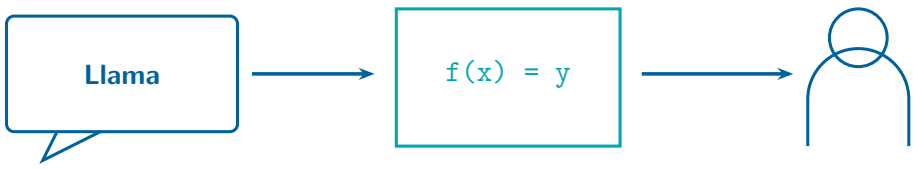
	state	evidence_ matches	engineering_ pass	scope_ matches	scope	acknowled ged	equipment_ dispatch
0	HOLD	True	True	False	MV phases only	False	False
1	READY_FOR_ READBACK	True	True	True	MV phases only	True	False

Reference L12 output. Run L12 here, after the three cases and the L10B replay.

- Full MV/LV request: HOLD. The explicit MV teaching request allows a simulated readback.
- A wrong recommendation ID is rejected. equipment_dispatch stays False.

Source: Classroom_Labs.ipynb, L12; cell classroom-033; simulated readback only.

Llama chooses; tools calculate; people decide



Our grid teaching adaptation of the language/checking distinction in Stanford Lecture 1, p.53.

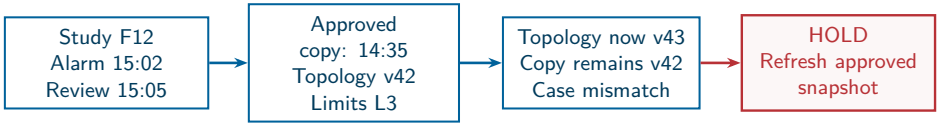
- The agent coordinates a study. Numerical tools supply the checks. The operator reviews the proposal.

Source: Monica Lam. CS224V Agentic AI, Lecture 1: Introduction. Stanford University, Fall 2026, p.53.

Start a control-room pilot with a clear study request

Proposed advisory pilot / teaching design

Illustrative F12 example; no actual utility data



The topology mismatch invalidates this copy for the requested study.

Proposed advisory pilot — teaching design

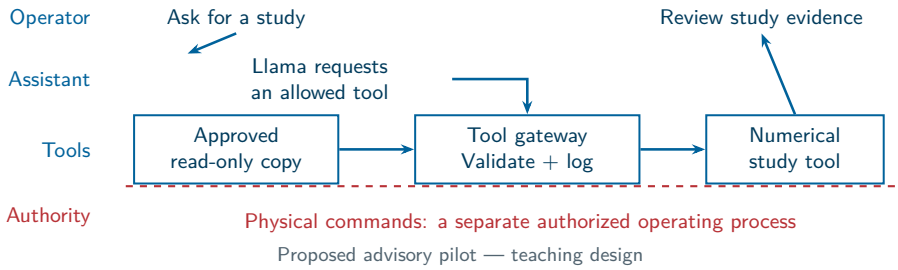
Proposed advisory pilot — teaching design. F12 is an illustrative identifier.

- Use an approved copy of the network data with a time stamp, topology version and limits.
- If required data are missing or too old, return HOLD and name the missing information.

Source: Teaching proposal informed by NIST OT guidance and the cited notebook checks.

Give the assistant a small, controlled tool list

Proposed advisory pilot / teaching design



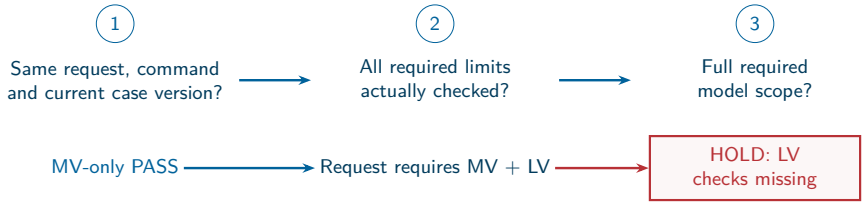
Proposed advisory pilot — teaching design. Classroom functions are not live control-room interfaces.

- An approved interface supplies read-only data and study tools. Log every request and result.
- Keep equipment commands in the utility's separate authorized operating process.

Source: Teaching proposal informed by NIST OT guidance and the cited notebook checks.

Check three things before returning a proposal

Proposed advisory pilot / teaching design



Proposed advisory pilot — teaching design

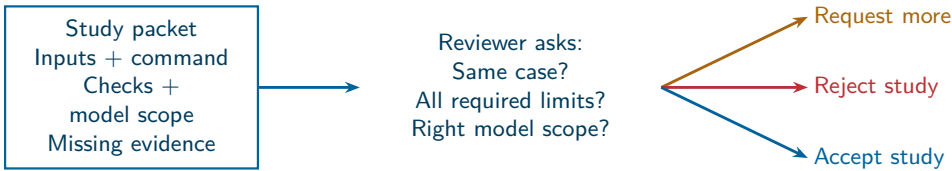
Proposed advisory pilot — teaching design. A PASS is limited to its tested model and inputs.

- Check that the inputs match, the required limits pass and the required network parts were modeled.
- Example: an MV-only PASS stays on HOLD when the request requires the full MV/LV network.

Source: Teaching proposal informed by NIST OT guidance and the cited notebook checks.

Give the operator a short evidence packet

Proposed advisory pilot / teaching design



Accepted study ≠ permission to operate equipment.

Proposed advisory pilot — teaching design

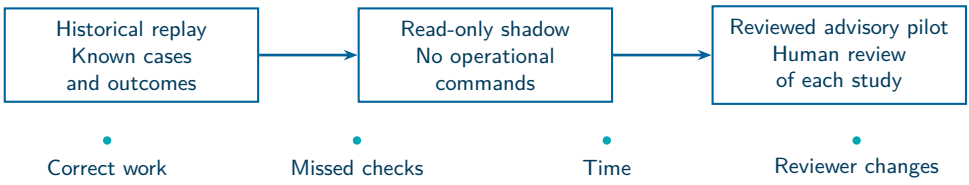
Proposed advisory pilot — teaching design. Review of a study is distinct from operating equipment.

- Show the input snapshot, selected option, worst margins, rejected trials and missing checks.
- The reviewer can ask for another study, reject the proposal or accept it for the next review step.

Source: Teaching proposal informed by NIST OT guidance and the cited notebook checks.

Adopt it in stages and measure the benefit

Proposed advisory pilot / teaching design



Compare with a fixed script + the same tools. The owner can stop the pilot.

Proposed advisory pilot — teaching design

Proposed advisory pilot — teaching design. Benefits must be measured against an appropriate baseline.

- Begin with past cases, then read-only shadow work, then a limited pilot with named reviewers.
- Compare against the same tools in a fixed script: correct studies, missed checks, time and rework.

Source: Teaching proposal informed by NIST OT guidance and the cited notebook checks.

Save a workshop record others can repeat

Get the Participant Lab Guide

- Save your changed inputs and results, labeled new, archived or simulated.
- Explain the request, tool call, result, checks and next decision.

Open the Full Reading Guide

Separate reading-guide PDF

Source: Setup: COLAB_GUIDE.md; Classroom_Labs.ipynb.